

# CS228 : Logic for Computer Science

## Module 1: Propositional Logic

### Lecture 2: Parse Trees, Semantics and Problems of Interest

Instructor: S. Akshay

IIT Bombay, India

# Logistics

## Updates

- ▶ Safe should be up and running
- ▶ Piazza is up and running - many interesting questions already!
- ▶ Problemsheets: This Thursday ...
- ▶ Optional Help Session: For a few students, already contacted!

# Module 1: Logic, the language/calculus for reasoning

A whole zoo of symbolic logics out there!

We will go from simple to more complicated

- ▶ Propositional Logic
- ▶ Predicate Logic
- ▶ Temporal logics
- ▶ Modal logics

## Recap: Propositional Logic

1. What are propositions?
2. Atomic propositions and how to combine them
3. Syntax of propositional logic: Propositional formulae

## Recall: Propositional formulas

Is this a proposition?

It is raining.

**Exercise 1.6:** Convert these to propositional logic

1. If it rains, you will get wet.
2. If it does not rain, you will not get wet.
3. If it rains, you will get wet unless you have an umbrella.
4. If it rains, you will get wet or you won't get wet.

## Topic 2.1

### Parse Trees

# Representing Propositional Formulae

A propositional formula is a **finite string** containing symbols in **AP** and logical connectives.

$F \in \mathbf{Prop}$  if

$$F ::= p \mid \top \mid \perp \mid \neg F \mid (F \vee F) \mid (F \wedge F) \mid (F \rightarrow F) \mid (F \leftrightarrow F) \mid (F \oplus F)$$

where  $p \in \mathbf{AP}$ .

## Representation of Propositional formulas as strings

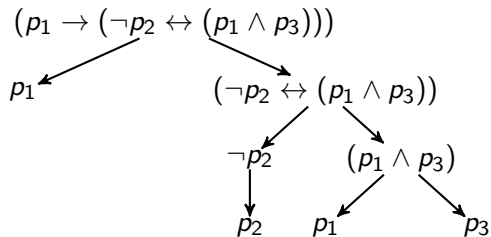
- ▶ Any  $F \in \mathbf{Prop}$  is a string over **AP** and logical connectives
- ▶ Two issues:
  1. Not all strings are formulas. e.g.,  $(\neg)()pq \rightarrow$
  2. If we don't bracket properly, we cannot parse it uniquely!
    - ▶ A partial solution : Preference order  $\neg$  then  $\vee, \wedge$  then  $\rightarrow, \leftrightarrow$ . So  $\neg p \vee q \rightarrow r$  is to be read as  $((\neg p \vee q) \rightarrow r)$ .
- ▶ A better solution: Tree representation!

# Representation as Parse trees

Definition: A **parse tree** of a formula  $F \in \mathbf{Prop}$  is a tree s.t.,

- ▶ the root is  $F$ ,
- ▶ leaves are atomic propositions, and
- ▶ each internal node is formed by applying some logical connective to its children.

Example



Theorem

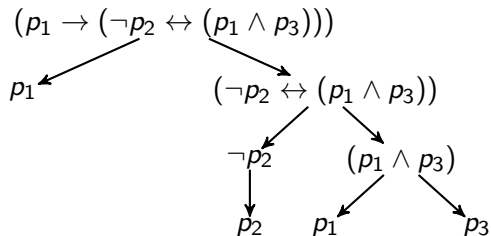
$F \in \mathbf{Prop}$  iff  $F$  has a parse tree

# Representation as Parse trees

Definition: A **parse tree** of a formula  $F \in \mathbf{Prop}$  is a tree s.t.,

- ▶ the root is  $F$ ,
- ▶ leaves are atomic propositions, and
- ▶ each internal node is formed by applying some logical connective to its children.

Example



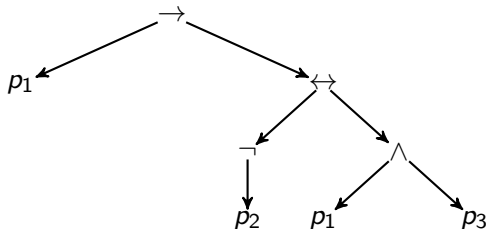
- ▶ Therefore, writing the tree allows to check that formula is well-formed.
- ▶ Formulas on internal nodes and leaves are called **sub-formulas**.

# Representation as Parse trees

Definition: A **parse tree** of a formula  $F \in \mathbf{Prop}$  is a tree s.t.,

- ▶ the root is  $F$ ,
- ▶ leaves are atomic propositions, and
- ▶ each internal node is formed by applying some logical connective to its children.

Example



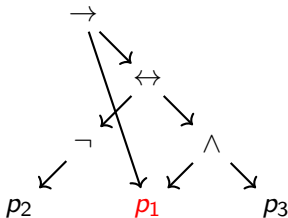
- ▶ Therefore, writing the tree allows to check that formula is well-formed.
- ▶ Formulas on internal nodes and leaves are called **sub-formulas**.
- ▶ Don't need brackets in the tree!

# Representation as Parse trees / DAGs

Definition: A **parse tree** of a formula  $F \in \mathbf{Prop}$  is a tree s.t.,

- ▶ the root is  $F$ ,
- ▶ leaves are atomic propositions, and
- ▶ each internal node is formed by applying some logical connective to its children.

Example



- ▶ Therefore, writing the tree allows to check that formula is well-formed.
- ▶ Formulas on internal nodes and leaves are called **sub-formulas**.
- ▶ Share leaves to get a directed-acyclic graph (DAG) instead of Tree!

# Some exercises

## Exercise 2.1

► Are these formulas in **Prop**? If so, draw the parse trees and dags for the formulas below:

1.  $(p \wedge (\neg q \rightarrow \neg p))$
2.  $\neg((\neg q \wedge (p \rightarrow r)) \wedge (r \rightarrow q))$
3.  $(s \vee ((\neg p) \rightarrow (\neg p)))$
4.  $(p \rightarrow (\neg q \wedge r \vee \neg p))$

# Where are we?

- ▶ So far: **Syntax** — which strings of symbols count as valid formulas.
- ▶ So far: **Parse Trees** — how to represent a formula's structure (and share sub-formulas via DAGs).
- ▶ Next: **Semantics** — given a well-formed formula, what does it actually *mean*? How do we compute its truth value?

## Topic 2.2

## Semantics

# Semantics

## What is the meaning of a formula?

- ▶ We start with truth values  $\mathcal{B} := \{0, 1\}$ , where we interpret 0 as False and 1 as True.
- ▶ Each atomic proposition  $p \in \mathbf{AP}$  is either True or False.  
e.g., “It is raining” is True right now (monsoon season!).
- ▶ Each propositional formula  $F \in \mathbf{Prop}$  is built from atomic propositions  
e.g., “It is raining  $\wedge$  I have an umbrella” is built from two atomic propositions.
- ▶ So “assigning” truth values to  $\mathbf{AP}$  gives “value” for every  $F \in \mathbf{Prop}$ .  
e.g., if both of the above are True, the formula “It is raining  $\wedge$  I have an umbrella” is True too.

## Definition

- ▶ An *assignment* is a map from  $\mathbf{AP} \rightarrow \mathcal{B}$ .
- ▶ A *Valuation or a model*  $m$  of a propositional formula  $F \in \mathbf{Prop}$  is an assignment of each atomic proposition in  $F$  to a truth value.

## Summarizing the truth values: Truth Tables

- ▶ For each assignment of variables, i.e., propositions, we obtain a value of a formula  $F$ .
- ▶ If we write down the values for ALL possible assignments, we get the “truth table”.
- ▶ Thus, the **semantics of  $F \in \mathbf{Prop}$** , denoted  $\llbracket F \rrbracket$ , is a map from  $\{0, 1\}^{\mathbf{AP}} \rightarrow \{0, 1\}$ .

What about  $G \rightarrow H$ ?

| $G$ | $H$ | $G \rightarrow H$ |
|-----|-----|-------------------|
| 1   | 1   | 1                 |
| 1   | 0   | 0                 |
| 0   | 1   | 1                 |
| 0   | 0   | 1                 |

- ▶ **Exercise 2.2:** Write down the truth tables for  $\vee$ ,  $\neg$ ,  $\leftrightarrow$ ,  $\oplus$ .

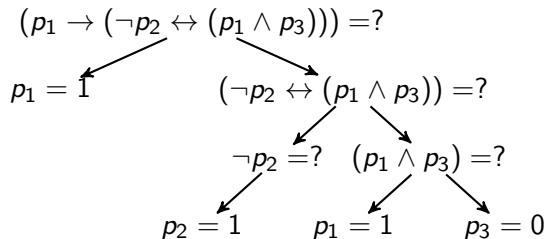
- ▶ Note that Truth table for True is just  $\begin{array}{|c|} \hline \top \\ \hline 1 \\ \hline \end{array}$  and False is  $\begin{array}{|c|} \hline \perp \\ \hline 0 \\ \hline \end{array}$ .

How do we generalize this for an arbitrary formula  $F$ ?

# Evaluating a formula using its Parse tree

## Question

If  $p_1 = 1$ ,  $p_2 = 1$ ,  $p_3 = 0$ , what is  $(p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$ ?



- ▶ For a given formula  $F$  and a given assignment, how long does it take to evaluate it? A. Linear time in size of the tree!
- ▶ But what is the size of the tree in worst case?

# Semantics of Propositional formula as Truth Tables

Consider:  $(p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$

| $p_1$ | $p_2$ | $p_3$ | $(p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$ |   |   |   |   |   |   |   |   |
|-------|-------|-------|-----------------------------------------------------------------|---|---|---|---|---|---|---|---|
| 0     | 0     | 0     | 0                                                               | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0     | 0     | 1     | 0                                                               | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0     | 1     | 0     | 0                                                               | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0     | 1     | 1     | 0                                                               | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 1 |
| 1     | 0     | 0     | 1                                                               | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 |
| 1     | 0     | 1     | 1                                                               | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1     | 1     | 0     | 1                                                               | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1     | 1     | 1     | 1                                                               | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 1 |

- ▶ Can think of it as going up the Parse tree.
- ▶ So we are applying an inductive evaluation!
  - ▶  $[F \wedge G] = 1$  iff  $[F] = 1$  and  $[G] = 1$
  - ▶  $[\neg G] = 1$  iff  $[G] = 0$
  - ▶ Complete the rest!

## Back to the example beyond good and evil

- ▶ In a country, every person is either good or evil (i.e., NOT good).
- ▶ Good people always tell the truth and evil people always lie.
- ▶ You meet two citizens  $A$  and  $B$ .  $A$  says, “I am evil or  $B$  is good”. What are  $A$  and  $B$ ?

### Using propositional logic

- ▶ Step 1: Introduce atomic propositions:  $p_A = A$  is good,  $p_B = B$  is good.
- ▶ Step 2: Encode puzzle using the propositional formula:  $F = \underbrace{(\neg p_A \vee p_B)}_{\text{Statement of A}} \leftrightarrow p_A$

### Exercise 2.3

Write out the Truth Table for  $F$

## Back to the example beyond good and evil

- ▶ In a country, every person is either good or evil (i.e., NOT good).
- ▶ Good people always tell the truth and evil people always lie.
- ▶ You meet two citizens  $A$  and  $B$ .  $A$  says, “I am evil or  $B$  is good”. What are  $A$  and  $B$ ?

### Using propositional logic

- ▶ Step 1: Introduce atomic propositions:  $p_A = A$  is good,  $p_B = B$  is good.
- ▶ Step 2: Encode puzzle using the propositional formula:  $F = \underbrace{(\neg p_A \vee p_B)}_{\text{Statement of A}} \leftrightarrow p_A$
- ▶ Step 3: Solve the puzzle: Is there a unique assignment of truth values to  $p_A$  and  $p_B$  under which  $[[F]] = 1$ ? How do we do this? **Check for 1 in Truth Table!**

| $p_A$ | $p_B$ | $\neg p_A$ | $\neg p_A \vee p_B$ | $F$ |
|-------|-------|------------|---------------------|-----|
| 0     | 0     | 1          | 1                   | 0   |
| 0     | 1     | 1          | 1                   | 0   |
| 1     | 0     | 0          | 0                   | 0   |
| 1     | 1     | 0          | 1                   | 1   |

## More examples

A prisoner must choose between three rooms. One room has a Lady and other two have tigers. If the prisoner chooses the room with the Lady, he wins! If he chooses a wrong room, he (probably) gets eaten by the tiger!

Each room has a sign in front of it. BUT Only one of the signs is true; the other two are false.

**(Room1)**    *The Lady is not in this room*

**(Room2)**    *The Lady is not in this room*

**(Room3)**    *The Lady is in Room 2*

Which room has the Lady?!

### Exercise 2.4

1. Take the encoding of this problem you had written last lecture. Write the Truth Table for that formula.
2. Use the Truth Table to solve the problem.

## Topic 2.3

### Problems of Interest

# Satisfiability

## Definition (Satisfiability)

$F \in \mathbf{Prop}$  is said to be **satisfiable** if

- ▶ there exists a valuation  $m$  of  $F$  such that  $m(F) = 1$ , i.e.,  $\llbracket F \rrbracket = 1$ .
- ▶ in other words, we say  $F$  has a *satisfying* assignment.

## Examples

- ▶ Country with Good and Bad people example
- ▶ Lady and the Tiger example
- ▶ Many many more!

How did we solve it? Check for 1 entry.

# Validity

Definition:  $F \in \mathbf{Prop}$  is said to be **valid** if

- ▶ for all valuations  $m$  of  $F$ , we have  $m(F) = 1$  i.e.,  $\llbracket F \rrbracket = 1$ .
- ▶ Valid formulas are also called **tautologies**.
- ▶ Can we read it from the Truth Table?! Is  $G = (p_1 \rightarrow (\neg p_2 \leftrightarrow (p_1 \wedge p_3)))$  valid?
- ▶ Just check if all entries in col for  $G$  are 1.

| $p_1$ | $p_2$ | $p_3$ | $G$ |
|-------|-------|-------|-----|
| 0     | 0     | 0     | 1   |
| 0     | 0     | 1     | 1   |
| 0     | 1     | 0     | 1   |
| 0     | 1     | 1     | 1   |
| 1     | 0     | 0     | 0   |
| 1     | 0     | 1     | 1   |
| 1     | 1     | 0     | 1   |
| 1     | 1     | 1     | 0   |

# Link between Satisfiability and Validity

What is the link between Satisfiability and Validity?

## Theorem

Let  $F \in \mathbf{Prop}$ .  $F$  is satisfiable iff  $\neg F$  is not valid.

Proof:

- ▶ ( $\implies$ ) Suppose  $F$  is sat. By defn,  $\exists$  valuation  $m$  of  $F$  such that  $m(F) = 1$ , but then  $m(\neg F) = 0$ . That is,  $\neg F$  is not valid.
- ▶ ( $\impliedby$ ) Conversely, suppose  $\neg F$  is not valid. There must exist valuation  $m'$  of  $\neg F$  such that  $m'(\neg F) = 0$ . Then  $m'(F) = 1$ , which implies that  $F$  is satisfiable.

In other words, if you have an algorithm to check for Satisfiability of propositional formulas, you can use it to check for validity of formulas and vice versa!

# Semantic Entailment

**Definition:** Consider propositional formulas  $F, G$  over **AP**.

Whenever  $F$  evaluates to 1,  $G$  also evaluates to 1 — we say,  $F \models G$ .

That is, for all assignments  $m$  of **AP**, if  $m(F) = 1$  then  $m(G) = 1$ .

**Exercise 2.5:** Do the following hold? Write True/False

▶  $p \wedge q \models p$

▶  $p \vee q \models q$

**Definition (Semantic Equivalence)**

We say that  $F$  and  $G$  are **semantically equivalent**, denoted  $F \equiv G$ , if  $F \models G$  and  $G \models F$ .

Example:  $(p \rightarrow q) \equiv (\neg p \vee q)$ .